Annexe B : Corrigés des exercices

Introduction

Au fil des divers TPs, des exercices vous ont été proposés pour tester votre compréhension des concepts abordés. Dans cette annexe, nous allons discuter une solution possible à chacun de ces exercices.

Exercices TP1

Pour créer un dépôt git, il suffit d'utiliser la commande `git init` au sein d'une arborescence de fichiers. On peut le faire au sein d'un dossier existant, ou bien créer un nouveau dossier pour le dépôt comme on l'a fait pendant le TP :

```
$ mkdir ~/Bureau/Exercice1
$ cd ~/Bureau/Exercice1
$ git init
```

Dans le monde du shell Unix, on désigne les fichiers portant l'extension « .o » par la syntaxe `*.o` qui signifie « tout fichier ou dossier dont le nom se termine par .o ». On peut, en utilisant cette syntaxe, dire à git d'ignorer les fichiers portant cette extension via le fichier spécial « .gitignore » :

```
$ echo '*.o' > .gitignore
```

Maintenant, créons un fichier :

```
$ echo 'Blabla' > Test.txt
```

On peut indiquer à git qu'on souhaite versionner ce fichier avec la commande `git add`. Ce fichier sera alors inclus dans toutes les versions de l'arborescence enregistrées ultérieurement. Nous verrons comment au TP2.

```
$ git add Test.txt
```

Pour déplacer un fichier en tenant git informé de cette opération, on peut utiliser la commande `git mv` :

```
$ git mv Test.txt AutreTest.txt
```

Si l'on regarde la sortie de `git status`...

```
$ git status
Sur la branche master

Aucun commit

Modifications qui seront validées :
  (utilisez "git rm --cached <fichier>..." pour désindexer)
        nouveau fichier : AutreTest.txt

Fichiers non suivis:
```

```
  (utilisez "git add <fichier>..." pour inclure dans ce qui sera validé)
        .gitignore
```

. . . on y trouve, outre l'information de branche actuelle qu'on choisit d'ignorer pour l'instant, les informations suivantes :

- Aucune version n'a été enregistrée pour l'instant.
- Si l'on enregistrait notre première version maintenant, le fichier « AutreTest.txt » en ferait partie, mais pas le fichier « .gitignore ».

Exercices TP2

La procédure pour créer un dépôt git vide est la même que précédemment :

```
$ mkdir ~/Bureau/Exercice2
$ cd ~/Bureau/Exercice2
$ git init
```

Ajoutons-y un peu de contenu :

```
$ echo 'Mon premier fichier' > Fichier1.txt
$ echo 'Mon deuxième fichier' > Fichier2.txt
$ echo 'Mon troisième fichier' > Fichier3.txt
```

On indique à git qu'on souhaite versionner des fichiers avec `git add`. Lorsqu'on souhaite demander à git de sauvegarder tous les fichiers du dépôt, un raccourci pratique est l'option `--all` :

```
$ git add --all
```

Ensuite, on peut créer un commit contenant tous ces fichiers avec `git commit` :

```
$ git commit -m 'Mon premier commit'
```

Maintenant, modifions deux fichiers. . .

```
$ echo 'De plus' >> Fichier1.txt
$ echo 'En plus' >> Fichier2.txt
```

Si l'on souhaite créer un commit qui ne contient que les modifications d'un seul fichier, il suffit de n'appliquer `git add` qu'à celui-ci :

```
$ git add Fichier1.txt
$ git commit -m 'Mon deuxième commit'
```

Du coup, il reste des modifications non sauvegardées à l'autre fichier :

```
$ git status
```

Comme ce fichier est sous gestion de version, on peut visualiser ses modifications non sauvegardées avec `git diff` :

```
$ git diff
```

Pour créer un commit comprenant toutes les modifications non sauvegardées, on a plusieurs possibilités. Nous avons déjà vu `git add --all`, quand tous les fichiers sont sous gestion de version on peut aussi utiliser `git commit -a` :

```
$ git commit -am 'Mon troisième commit'
```

À ce stade, nous avons créé trois commits dans le dépôt. Le plus récemment créé peut être désigné par `HEAD`, son prédécesseur par `HEAD~`, et le commit précédent, qui est le premier commit de l'historique, par `HEAD~2`.

Nous pouvons donc afficher la différence entre les deux premiers commits de l'historique avec la commande...

```
$ git diff HEAD~2 HEAD~
```

Le répertoire de travail est-il bien propre ?

```
$ git status
```

Il l'est. On peut donc partir en voyage dans l'historique, par exemple pour visiter le grand-parent du commit actuel, `HEAD~2` :

```
$ git checkout HEAD~2
```

Si, durant notre visite, on se rend compte qu'on pourrait avoir besoin de revenir sur ce commit ultérieurement, il peut être judicieux d'y attacher une étiquette avec la commande `git tag` :

```
$ git tag papy
```

On peut revenir sur la branche master avec la commande `git checkout` :

```
$ git checkout master
```

Exercices TP3

Un dépôt git est un dossier comme les autres, on peut en créer une copie avec une copie récursive :

```
$ cp -r ~/Bureau/MonDepot ~/Bureau/MonAutreDepot
$ cd ~/Bureau/MonAutreDepot
```

Le parent du commit actif est désigné par la syntaxe `HEAD~`, et on peut le visiter avec la commande `git checkout` :

```
$ git checkout HEAD~
```

Ensuite, on peut créer une branche qui en part avec `git branch` :

```
$ git branch nouvelle-branche
```

Et on peut basculer sur cette branche avec `git checkout` :

```
$ git checkout nouvelle-branche
```

Pour créer un commit ajoutant un nouveau fichier, on crée le fichier, puis on le l'ajoute à l'index avec `git add`, et on crée le commit avec `git commit` :

```
$ echo 'Un fichier' > LeFichier.txt
$ git add LeFichier.txt
$ git commit -m "Ajout d'un fichier"
```

On peut ensuite visualiser la branche master et la branche que l'on vient de créer en lançant l'outil gitg dans le dépôt...

```
$ gitg &
```

...ou en rafraîchissant son affichage avec Ctrl+R s'il est déjà ouvert. N'oubliez pas de sélectionner l'option « Branches » dans le menu de droite pour pouvoir visualiser les deux branches simultanément.

Notre branche a un peu de retard avec la branche master, nous pouvons rattraper ce retard en y fusionnant les nouveautés de la branche master avec `git merge` :

```
$ git merge master
```

Maintenant, remplaçons le contenu d'un fichier au sein de la branche active...

```
$ echo 'v1' > LeFichier.txt
$ git add LeFichier.txt
$ git commit -m 'Première version'
```

...et au sein de la branche master, en basculant sur celle-ci :

```
$ git checkout master
$ echo 'v2' > LeFichier.txt
$ git add LeFichier.txt
$ git commit -m 'Deuxième version'
```

Si l'on essaye de fusionner nouvelle-branche au sein de master avec `git merge`, il y aura conflit au niveau de ce fichier :

```
@ git merge nouvelle-branche
```

On peut résoudre ce conflit de plusieurs façons, l'approche préconisée dans cette formation est l'utilisation d'un outil de visualisation tel que Meld :

```
$ meld .
```

Au sein de l'interface de Meld, double-cliquez sur le fichier conflictuel, qui apparaît en rouge dans l'interface, puis résolvez le conflit en choisissant la version « de gauche », la version « de droite », ou en écrivant une version de votre cru qui vous semble réunir les meilleures idées de chacune.

Après avoir quitté Meld en sauvegardant vos changements et en validant la résolution du conflit, vous pourrez terminer la fusion avec `git merge --continue` ou `git commit` :

```
$ git merge --continue
```

Exercices TP4

Pour cloner un dépôt, on utilise la commande `git clone` :

```
$ git clone ~/Bureau/MonDepot ~/Bureau/TonDepot
```

Si l'on compare le contenu de l'original et du dépôt du point de vue du répertoire de travail, avec la commande `ls -al`...

```
$ cd ~/Bureau/MonDepot/
$ ls -al
$ cd ../TonDepot/
$ ls -al
```

... on observe les différences suivantes :

- Le dossier MonDossier n'est pas présent dans le clone.
- Le fichier Poubelle.log n'est pas présent dans le clone.

Ces différences s'expliquent par le fait que le répertoire de travail d'un clone n'est pas une copie à l'identique de celui du dépôt d'origine, mais représente plutôt une copie du dernier *commit* enregistré dans le dépôt d'origine.

Ce *commit* ne contenait pas le dossier MonDossier puisqu'il s'agit d'un dossier vide, non géré par git, ni le fichier Poubelle.log puisqu'il est ignoré par git.

Si maintenant l'on compare les dépôts du point de vue de l'arbre de *commits* avec l'aide de l'utilitaire `gitg`...

```
$ gitg &
$ cd ../MonDepot
$ gitg &
```

On constate que leur structure générale est similaire mais que le clone possède le dépôt original comme *remote* (sous le nom par défaut `origin`), et visualise simultanément sa version locale de la branche `master` et une copie locale de celle du dépôt original, `origin/master`.

C'est le fait de pouvoir différentier les versions locales et distantes de branches, comme ici `master`, qui permet d'utiliser git de façon distribuée sans connexion permanente à internet.

Pour créer une branche en vue d'y ajouter quelques commits, on peut utiliser le raccourci `git checkout -b` :

```
$ git checkout -b test
$ echo 'Le contenu' > AutreContenu.txt
$ git commit -am 'Un commit de test'
$ echo 'Plus de contenu' >> AutreContenu.txt
$ git commit -am 'Un autre commit de test'
```

À ce stade, le clone du dépôt n'est pas au courant de l'existence de ces commits et de cette nouvelle branche, puisqu'en git les échanges entre dépôts sont déclenchés

manuellement. On peut remédier à cela avec `git fetch` :

```
$ cd ../TonDepot
$ git fetch origin
```

Maintenant, le clone `TonDepot` est au courant de l'existence de la branche `test` du dépôt `MonDepot`, qu'il représente comme la branche distante `origin/test`. Nous pouvons en créer une version locale et basculer dessus avec un raccourci basé sur `git checkout` :

```
$ git checkout test
```

Ensuite, nous pouvons rajouter des commits à cette branche locale, comme nous le ferions sur n'importe quelle autre branche locale :

```
$ echo "Renouvelons l'offre" > NouveauFichier.txt
$ git add NouveauFichier.txt
$ git commit -m "Ajout d'un fichier"
$ echo 'Renouvelons la demande' >> NouveauFichier.txt
$ git commit -am "Complétion d'un fichier"
```

Pour récupérer ces changements dans le dépôt d'origine, nous allons devoir ajouter notre clone comme *remote* de ce dernier...

```
$ cd ../MonDepot
$ git remote add copieur ~/Bureau/TonDepot
```

... puis récupérer les changements de sa branche `test` et les intégrer au sein de notre branche test locale avec, par exemple, le raccourci `git pull` :

```
$ git pull copieur test
```

Pour terminer, ajoutons quelques commits sur la branche `master` de `MonDepot` :

```
$ git checkout master
$ echo 'Je ne sais quoi inventer' > Incertitude.txt
$ git add --all
$ git commit -m 'Méditation'
$ echo 'Je ne sais quoi penser' >> Incertitude.txt
$ git add Incertitude.txt
$ git commit -am 'Moment philosophique'
```

On peut propager ces changements sur la branche `master` de `TonDepot` avec un simple `git push` :

```
$ git push copieur master
```

Si l'on souhaite intégrer les changements correspondants à la branche `test` du clone, il faudra comme précédemment s'y rendre, récupérer les changements, et les fusionner. Faisons cela avec `git fetch` et `git merge` :

```
$ cd ../TonDepot
$ git fetch origin
```

```
$ git merge origin/master
```

Exercices TP5

Tout d'abord, rendons-nous dans notre clone du dépôt « ddhc » :

```
$ cd ~/Bureau/ddhc
```

On peut vérifier la configuration des *remotes* avec `git remote -v` :

```
$ git remote -v
```

Ici, nous suivons la configuration habituelle : `origin` pointe sur le *fork*, et `upstream` sur le dépôt officiel.

Pour créer une nouvelle branche partant de la branche `master`, on commence par basculer sur cette branche...

```
$ git checkout master
```

... puis on crée la nouvelle branche et bascule dessus...

```
$ git checkout -b copyright
```

... et on peut y enregistrer des changements de la façon habituelle :

```
$ echo -e '\n(C) 1789, Être Suprême Inc.' >> README.md
$ git commit -am "Ajout d'une notice de copyright"
```

On peut ensuite pousser la branche sur son *fork* avec `git push`. La première fois, on doit spécifier vers quelle *remote* on pousse, mais on peut mémoriser cette association pour la suite avec `--set-upstream` :

```
$ git push --set-upstream origin copyright
```

On peut ensuite répéter l'opération autant de fois qu'on le souhaite, sans nécessité de répéter le nom de la *remote* à chaque fois dans `git push` :

```
$ echo 'AAAAAAH' >> README.md
$ git commit -am 'Panique à bord !'
$ echo 'QQQQQQH' >> README.md
$ git commit -am 'Panique à bord en QWERTY !'
$ git push
```

Il est possible qu'entre-temps le `master` du dépôt officiel ait évolué. Vérifions ce point en mettant à jour notre `master` local :

```
$ git checkout master
$ git pull --ff-only
```

Un dépôt distant fonctionne comme une copie de sauvegarde. Même si on supprimait malencontreusement une branche locale...

```
$ git branch -D copyright
```

. . . on pourrait toujours la récupérer à partir de la version sauvegardée sur le dépôt distant, via le raccourci habituel :

```
$ git checkout copyright
```